Amplify GenAI - Functional Specifications

Version: 0.9.0
Date: 2026-03-01
Authors: Vanderbilt University (Jules White, Allen Karns, Karely Rodriguez, Max Moundas)

Table of Contents

1. System Overview
2. Architecture Summary
3. Module Inventory
4. 3.1 Frontend Modules
5. 3.2 Backend Services
6. Core User Journeys
7. Module Specifications
8. Cross-Module Integration Map
9. Data Model
10. File-to-Functionality Tag Map

1. System Overview

Amplify GenAI is a full-stack, enterprise-grade conversational AI platform that enables users to interact with multiple Large Language Models (LLMs), upload and query documents via Retrieval-Augmented Generation (RAG), create and share custom AI assistants, execute multi-step workflows, and collaborate through a marketplace of prompts and assistants.

Key Capabilities

Capability	Description
Multi-Model Chat	Streaming conversations with GPT-4, Claude 3, Mistral via Azure OpenAI and AWS Bedrock
RAG (Retrieval-Augmented Generation)	Upload documents, auto-chunk, embed, and query them semantically during chat
Custom Assistants	Create, configure, and share AI assistants with specialized instructions and attached knowledge
Structured Output	Generate JSON, CSV, and function-call responses from natural language
Workflow Engine	Define and execute multi-step LLM pipelines
Marketplace	Browse, publish, and share prompts, assistants, and automation templates
Collaboration	Share conversations, assistants, and files with fine-grained access control
Multi-Language	21 locale support (en, es, fr, de, ja, ko, zh, ar, etc.)
Enterprise Security	AWS Cognito OAuth 2.0, JWT validation, per-object RBAC, data disclosure compliance
Usage Tracking	Token-level billing, model cost comparison, per-user accounting

2. Architecture Summary

```
                          +-----------------------+
                          |    Frontend (Next.js)  |
                          |   React 18 + TypeScript|
                          |   NextAuth + Cognito   |
                          +-----------+-----------+
                                      |
                          HTTPS / API Gateway / Lambda URLs
                                      |
          +---------------------------+---------------------------+
          |                           |                           |
+--------v--------+     +------------v-----------+    +-----------v-----------+
| amplify-lambda   |     | amplify-lambda-js      |    | amplify-assistants     |
| (Python 3.11)    |     | (Node.js 18, HTTP/2)   |    | (Python 3.11)          |
| REST API:        |     | Streaming Chat:        |    | OpenAI Assistants:     |
| - Files          |     | - Azure OpenAI (GPT)   |    | - Create/List/Share    |
| - Assistants     |     | - Bedrock (Claude)     |    | - Code Interpreter     |
| - Market         |     | - Bedrock (Mistral)    |    | - Thread Management    |
| - State          |     | - RAG Integration      |    +----------+------------+
| - RAG Pipeline   |     | - Workflow Engine      |               |
| - Accounts       |     | - Token Billing        |               |
+--------+---------+     +------------+-----------+               |
          |                           |                           |
          +---------------------------+---------------------------+
                                      |
              +----------------------+----------------------+
              |                      |                      |
    +---------v--------+   +---------v--------+   +----------v---------+
    |    embedding      |   |   chat-billing   |   |   object-access    |
    | (Python 3.11)     |   | (Python 3.11)    |   | (Python 3.11)      |
    | - Vector Embed    |   | - Usage Tracking |   | - RBAC Permissions |
    | - Dual Retrieval  |   | - Exchange Rates |   | - Cognito Users    |
    | - pgvector/Aurora |   | - Cost Calc      |   | - Object ACLs      |
    +-------------------+   +------------------+   +--------------------+
```

Tech Stack:

Layer	Technology
Frontend	Next.js 14, React 18, TypeScript, Tailwind CSS, Mantine UI
Authentication	NextAuth.js + AWS Cognito (OAuth 2.0 / JWT)
Chat Streaming	Node.js Lambda with HTTP/2 Response Streaming
REST APIs	Python 3.11 Lambda + API Gateway
Vector DB	Amazon Aurora PostgreSQL with pgvector
Object Storage	Amazon S3 (documents, state, traces)
Metadata DB	Amazon DynamoDB (on-demand)
Queue/Async	Amazon SQS + S3 Event Notifications
LLM Providers	Azure OpenAI, AWS Bedrock (Anthropic, Mistral)
IaC	Serverless Framework v3 + CloudFormation

3. Module Inventory

3.1 Frontend Modules

#	Module	Location	Core Function
F1	**Home / App Shell**	`pages/api/home/`	Main application shell; loads models, conversations, auth state; manages global context
F2	**Chat Interface**	`components/Chat/`	Message display, input, streaming, model/temp selection, system prompt config
F3	**Chat Sidebar (Chatbar)**	`components/Chatbar/`	Conversation list, search, folder organization, new chat creation
F4	**Prompt Sidebar (Promptbar)**	`components/Promptbar/`	Prompt library, template management, variable prompts
F5	**Tab Sidebar**	`components/TabSidebar/`	Tab-based navigation across Chat, Marketplace, Share, Settings, Workspace
F6	**Assistants**	`components/Assistants/`	Assistant selector, configuration UI, provider switching (OpenAI/ Amplify)
F7	**Marketplace**	`components/Market/`	Browse, publish, and install community prompts, assistants, automations
F8	**Sharing**	`components/Share/`	Share conversations/assistants with other users; view received shares
F9	**Data Sources**	`components/DataSources/`	File attachment selector for RAG; tag-based filtering
F10	**Settings**	`components/Settings/`	User preferences (theme, storage, model defaults)
F11	**Workspace**	`components/Workspace/`	Workspace metadata management
F12	**Download/Export**	`components/Download/`	Export conversations to JSON
F13	**API Proxy Layer**	`pages/api/`	34 Next.js API routes proxying to backend Lambda functions
F14	**Services Layer**	`services/`	18 service modules for backend communication (chat, files, assistants, etc.)
F15	**Custom Hooks**	`hooks/`	8 React hooks for chat routing, state management, analytics
F16	**State Management**		

#	Module	Location	Core Function
		`pages/api/home/home.context.tsx`, `home.state.tsx`	React Context + Reducer for global application state
F17	**Type Definitions**	`types/`	27 TypeScript type files defining all domain entities
F18	**Utilities**	`utils/app/`	24 utility modules (compression, tokens, prompts, CSV, storage, etc.)
F19	**Internationalization**	`public/locales/`, `next-i18next.config.js`	21-locale translation files
F20	**Authentication**	`pages/api/auth/[...nextauth].js`	NextAuth.js with AWS Cognito provider, JWT refresh

3.2 Backend Services

#	Service	Location	Core Function
B1	**amplify-lambda**	`amplify-lambda/`	Python REST API: file management, assistant metadata, marketplace, state, RAG pipeline, accounts, document conversion
B2	**amplify-lambda-js**	`amplify-lambda-js/`	Node.js streaming chat: multi-model LLM calls, RAG context injection, workflow execution, token billing
B3	**amplify-assistants**	`amplify-assistants/`	Python REST API: OpenAI Assistants API integration, assistant CRUD, code interpreter, thread management
B4	**embedding**	`embedding/`	Python service: vector embedding generation (OpenAI/Bedrock), pgvector storage, hybrid dual-retrieval search
B5	**chat-billing**	`chat-billing/`	Python service: DynamoDB Streams consumer for usage tracking, model exchange rates, cost calculation
B6	**object-access**	`object-access/`	Python REST API: fine-grained per-object RBAC, Cognito user lookup, permission simulation
B7	**utilities**	`utilities/`	Python REST API: batch operations (chat renaming)

4. Core User Journeys

Journey 1: Multi-Model Conversational Chat

Description: User sends a message, selects an LLM model, and receives a streaming response.

Flow:

```
User types message in ChatInput (F2)
  → useSendService hook (F15) prepares ChatBody
    → Detects mode prefix: chat(), json(), csv(), fn()
    → chatService.sendChatRequestWithDocuments (F14)
      → POST to chat endpoint (F13 → B2)
        → router.js validates JWT, selects model (B2)
          → LLM class routes to Azure OpenAI / Bedrock Claude / Bedrock Mistral
            → HTTP/2 streaming response
              → SSE events parsed incrementally (F14)
                → Message tokens rendered in real-time (F2)
                  → Usage recorded in DynamoDB (B2 → B5)
                    → Conversation saved locally or to cloud (F18)
```

Modules Involved: F2, F14, F15, F16, F18 → B2, B5

Tagged Files:

File	Role
`frontend/components/Chat/ChatInput.tsx`	User message input and submission
`frontend/components/Chat/Chat.tsx`	Chat interface orchestration
`frontend/hooks/useChatSendService.ts`	Message routing and mode detection
`frontend/hooks/useChatService.ts`	Chat request execution
`frontend/services/chatService.ts`	HTTP streaming client to backend
`frontend/utils/app/responseWrapper.ts`	SSE stream parsing
`frontend/utils/app/conversation.ts`	Conversation persistence
`backend/amplify-lambda-js/index.js`	Lambda entry point (HTTP/2 streaming)
`backend/amplify-lambda-js/router.js`	Request routing, auth, model selection
`backend/amplify-lambda-js/common/llm.js`	LLM abstraction layer
`backend/amplify-lambda-js/azure/openai.js`	Azure OpenAI GPT integration
`backend/amplify-lambda-js/bedrock/anthropic.js`	AWS Bedrock Claude integration
`backend/amplify-lambda-js/bedrock/mistral.js`	AWS Bedrock Mistral integration
`backend/amplify-lambda-js/common/accounting.js`	Token usage recording
`backend/amplify-lambda-js/models/models.js`	Model registry and pricing

Journey 2: Document Upload and RAG Query

Description: User uploads a document, which is processed into vector embeddings, then queries it during chat for context-aware answers.

Flow:

```
A) Document Upload:
User selects file in DataSources (F9)
  → fileService.uploadFile (F14)
    → POST /assistant/files/upload (F13 → B1)
      → Presigned S3 URL generated (B1)
        → File uploaded directly to S3
          → S3 event triggers queue_document_for_rag (B1)
            → SQS → process_document_for_rag (B1)
              → Text extracted (PDF/DOCX/PPTX/XLSX handlers)
                → Text saved to S3 file-text bucket
                  → S3 event triggers queue_document_for_chunking (B1)
                    → SQS → process_document_for_chunking (B1)
                      → Chunks saved to S3 rag-chunks bucket
                        → S3 event triggers queue_document_for_embedding (B4)
                          → SQS → process_chunk_for_embedding (B4)
                            → Vector embeddings generated (OpenAI ada / Bedrock Titan)
                              → Stored in PostgreSQL pgvector (B4)

B) RAG Query During Chat:
User sends message with attached data sources (F2 + F9)
  → chatService sends dataSources in ChatBody (F14)
    → B2 resolves data sources, checks permissions
      → getContextMessages calls RAG endpoint (B2 → B4)
        → embedding-dual-retrieval (B4)
          → Hybrid semantic + BM25 keyword search
            → Top-k relevant chunks returned
              → Injected as context into LLM prompt
                → LLM generates context-aware response
                  → Sources metadata streamed to client
```

Modules Involved: F2, F9, F14 → B1, B2, B4, B6

Tagged Files:

File	Role
`frontend/components/DataSources/`	File selector UI for attaching documents
`frontend/services/fileService.ts`	File upload/download/query client
`frontend/pages/api/files/upload.ts`	API proxy for file upload
`frontend/pages/api/files/query.ts`	API proxy for file listing
`backend/amplify-lambda/assistant/assistant.py`	Presigned URL generation, file metadata CRUD
`backend/amplify-lambda/rag/core.py`	Document processing pipeline orchestration
`backend/amplify-lambda/rag/handlers/pdf.py`	PDF text extraction
`backend/amplify-lambda/rag/handlers/docx.py`	Word document text extraction
`backend/amplify-lambda/rag/handlers/pptx.py`	PowerPoint text extraction
`backend/amplify-lambda/rag/handlers/xlsx.py`	Excel text extraction
`backend/amplify-lambda/rag/handlers/csv.py`	CSV text extraction
`backend/amplify-lambda/rag/chunk/`	Document chunking (per format)
`backend/embedding/embedding-sqs.py`	Queue documents for embedding
`backend/embedding/embedding.py`	Generate vector embeddings (SQS worker)
`backend/embedding/embedding-dual-retrieval.py`	Hybrid semantic + keyword search endpoint
`backend/embedding/create_table.py`	PostgreSQL pgvector schema init
`backend/amplify-lambda-js/common/chat/rag/rag.js`	RAG query execution from chat context
`backend/amplify-lambda-js/datasource/datasources.js`	Data source resolution and access control

Journey 3: Assistant Creation and Usage

Description: User creates a custom AI assistant with specific instructions and attached files, then uses it in conversations.

Flow:

```
A) Create Assistant:
User opens Assistants panel (F6)
  → Fills in name, description, instructions, selects tools
    → Attaches files / data sources
      → assistantService.createAssistant (F14)
        → POST /assistant/create (F13 → B1 or B3)
          → Assistant definition stored in DynamoDB
            → File associations created
              → Object permissions set (B6)

B) Use Assistant in Chat:
User selects assistant from dropdown (F6)
  → selectedAssistant set in HomeContext (F16)
    → Message sent with assistantId in options
      → B2 receives request with assistant context
        → chooseAssistantForRequest loads assistant instructions (B2)
          → Assistant instructions injected as system prompt
            → Attached data sources automatically included in RAG
              → Response generated with assistant persona
```

Modules Involved: F2, F6, F14, F16 → B1, B2, B3, B6

Tagged Files:

File	Role
`frontend/components/Assistants/`	Assistant selector and configuration UI
`frontend/services/assistantService.ts`	Assistant CRUD client
`frontend/pages/api/assistant/op.ts`	API proxy for assistant operations
`frontend/types/assistant.ts`	Assistant type definitions
`frontend/utils/app/assistants.ts`	Assistant utility functions
`backend/amplify-lambda/assistant/assistant.py`	Assistant metadata and file management
`backend/amplify-assistants/service/core.py`	Core assistant operations
`backend/amplify-assistants/openaiazure/assistant.py`	OpenAI Assistants API integration
`backend/amplify-assistants/openaiazure/assistant_api.py`	Raw OpenAI API calls
`backend/amplify-lambda-js/assistants/assistants.js`	Assistant selection in chat routing
`backend/object-access/object_access.py`	Permission management for shared assistants

Journey 4: Marketplace Browse, Publish, and Install

Description: User browses community-shared prompts and assistants, publishes their own, or installs items from the marketplace.

Flow:

```
A) Browse Marketplace:
User clicks Market tab (F5 → F7)
  → marketService.getCategories (F14)
    → POST /market/category/list (F13 → B1)
      → DynamoDB query for categories
        → Categories displayed in Market UI (F7)
          → User selects category
            → marketService.getItems (F14)
              → POST /market/category/get (F13 → B1)
                → Items listed with descriptions

B) Publish to Marketplace:
User creates a prompt/assistant
  → Clicks "Publish to Market" (F7)
    → marketService.publishItem (F14)
      → POST /market/item/publish (F13 → B1)
        → Item saved to DynamoDB market-items table
          → DynamoDB Stream triggers index_market_item (B1)
            → Search index updated in S3

C) Install from Marketplace:
User selects marketplace item
  → Item details displayed (F7)
    → User clicks Install
      → Prompt/assistant/workflow imported into local state (F16, F18)
```

Modules Involved: F5, F7, F14, F16, F18 → B1

Tagged Files:

File	Role
`frontend/components/Market/Market.tsx`	Marketplace browse/publish UI
`frontend/services/marketService.ts`	Marketplace API client
`frontend/pages/api/market/op.ts`	API proxy for market operations
`frontend/types/market.ts`	Market item type definitions
`backend/amplify-lambda/market/market.py`	Marketplace CRUD and search
`backend/amplify-lambda/market/market_ideator.py`	AI-powered item ideation
`backend/amplify-lambda/market/rebuild_index.py`	Search index rebuilding

Journey 5: Conversation Sharing and Collaboration

Description: User shares a conversation with colleagues, who can then view and continue from the shared context.

Flow:

```
User opens conversation (F2)
  → Clicks Share button (F8)
    → ShareAnythingModal opens
      → User enters recipient email(s) (autocomplete via F14)
        → shareService.shareConversation (F14)
          → POST /share/share (F13 → B1)
            → Shared item stored in S3 + DynamoDB
              → Object permissions created (B1 → B6)

Recipient:
  → Sees shared item in Shared Items tab (F5 → F8)
    → shareService.getShared (F14)
      → POST /share/shared (F13 → B1)
        → Shared conversation loaded
          → Can view and continue chat from shared context
```

Modules Involved: F2, F5, F8, F14 → B1, B6

Tagged Files:

File	Role
`frontend/components/Share/ShareAnythingModal.tsx`	Sharing dialog UI
`frontend/components/Share/SharedItemList.tsx`	Received shares list
`frontend/services/shareService.ts`	Sharing API client
`frontend/services/emailAutocompleteService.ts`	Email autocomplete for recipient selection
`frontend/pages/api/share/share.ts`	API proxy for sharing
`frontend/pages/api/share/shared.ts`	API proxy for received shares
`backend/amplify-lambda/state/share.py`	Server-side sharing logic
`backend/object-access/object_access.py`	Permission grants for shared objects
`backend/object-access/cognito_users.py`	User lookup for sharing targets

Journey 6: Structured Output (JSON / CSV / Function Calling)

Description: User generates structured data outputs from natural language using special prompt prefixes.

Flow:

```
User types message with prefix:
  json()   → Loose JSON output
  json!()  → Strict JSON with schema validation
  csv()    → Tabular CSV output
  fn()     → Function calling with defined schema

  → useSendService detects prefix via getPrefix (F15)
    → Routes to specialized handler:
      → sendJsonChatRequest (for json())
      → sendJsonChatRequestWithSchema (for json!())
      → sendCSVChatRequest (for csv())
      → sendFunctionChatRequest (for fn())
    → ChatBody configured with response_format / functions (F14)
      → Backend processes with model-appropriate formatting (B2)
        → Response parsed and rendered in structured view (F2)
```

Modules Involved: F2, F14, F15 → B2

Tagged Files:

File	Role
`frontend/hooks/useChatSendService.ts`	Prefix detection and routing logic
`frontend/hooks/useChatService.ts`	Specialized request builders (JSON, CSV, fn)
`frontend/utils/app/csv.ts`	CSV schema generation
`frontend/utils/app/incrementalCsvParser.ts`	Streaming CSV parser
`frontend/types/chat.ts`	ChatBody, CustomFunction, JsonSchema types
`backend/amplify-lambda-js/common/llm.js`	LLM abstraction with function calling support
`backend/amplify-lambda-js/common/chat/events/openaifn.js`	Function calling event transforms

Journey 7: Workflow Execution

Description: User defines and runs multi-step LLM workflows that chain prompts together.

Flow:

```
User creates WorkflowDefinition (F4 or F7)
  → Defines steps with inputs, outputs, and LLM calls
    → Saves to local storage or imports from marketplace
      → User triggers workflow execution
        → useOpsService.executeWorkflow (F15)
          → Request sent to B2 with workflow body
            → workflow.js processes steps sequentially (B2)
              → Each step calls LLM with previous step's output
                → Status events streamed back (progress indicators)
                  → Final output returned to user (F2)
```

Modules Involved: F2, F4, F15 → B2

Tagged Files:

File	Role
`frontend/types/workflow.ts`	WorkflowDefinition, Status types
`frontend/utils/app/workflows.ts`	Workflow persistence
`frontend/hooks/useOpsService.ts`	Workflow execution client
`backend/amplify-lambda-js/workflow/workflow.js`	Multi-step workflow engine
`backend/amplify-lambda-js/workflow/workflowCreator.js`	Workflow schema validation

Journey 8: Authentication and Session Management

Description: User authenticates via AWS Cognito, obtains a session, and maintains authenticated state across the application.

Flow:

```
User navigates to application
  → NextAuth checks session (F20)
    → If no session: redirect to Cognito login
      → User authenticates via Cognito hosted UI
        → OAuth callback with tokens
          → NextAuth JWT callback stores access/refresh tokens
            → Session created (59-minute max age)
              → Access token injected into all API requests (F14)
                → Backend validates JWT on every request (B1, B2, B3)
                  → User identity extracted from token claims

Token Refresh:
  → Session approaching expiry
    → NextAuth JWT callback attempts token refresh
      → If refresh fails: user redirected to re-authenticate
```

Modules Involved: F13, F14, F20 → B1, B2, B3

Tagged Files:

File	Role
`frontend/pages/api/auth/[...nextauth].js`	NextAuth configuration with Cognito provider
`frontend/pages/_app.tsx`	SessionProvider wrapper
`frontend/services/chatService.ts`	Bearer token injection
`backend/amplify-lambda/common/validate.py`	@validated decorator for JWT verification
`backend/amplify-lambda/common/permissions.py`	Role-based access and claim extraction
`backend/amplify-lambda-js/common/params.js`	JWT extraction from request headers

Journey 9: Cloud Conversation Sync

Description: User stores conversations in the cloud (S3) for cross-device access and backup.

Flow:

```
User selects "Cloud Storage" in settings (F10)
  → storageSelection set in state (F16)
    → On conversation update:
      → conversationWithCompressedMessages compresses messages (F18)
        → remoteConversationService.uploadConversation (F14)
          → POST /conversation/remote/op (F13 → B1)
            → Conversation JSON stored in S3 state bucket (B1)

On app load (different device):
  → home.tsx checks for remote storage (F1)
    → updateWithRemoteConversations (F18)
      → Fetches conversation metadata from S3
        → Downloads and decompresses messages on demand
          → Local state synced with cloud state
```

Modules Involved: F1, F10, F14, F16, F18 → B1

Tagged Files:

File	Role
`frontend/services/remoteConversationService.ts`	Cloud sync upload/download client
`frontend/pages/api/conversation/remote/op.ts`	API proxy for remote conversation ops
`frontend/utils/app/conversationStorage.ts`	Storage type detection (local vs cloud)
`frontend/utils/app/conversation.ts`	Compression/decompression utilities
`frontend/utils/app/messages.ts`	LZW message compression
`frontend/utils/app/lzwCompression.ts`	LZW compression algorithm
`backend/amplify-lambda/state/share.py`	S3-based state persistence

Journey 10: Usage Tracking and Billing

Description: System tracks token usage per request, calculates costs by model, and maintains per-user billing records.

Flow:

```
LLM response completes (B2)
  → accounting.js records usage (B2)
    → Token counts (input + output) written to chat-usage DynamoDB table
      → DynamoDB Stream triggers chat-billing Lambda (B5)
        → Cost calculated: tokens x model rate (B5)
          → Exchange rates updated (B5)
            → Billing record written to billing table
              → Account balance updated (B1)

Frontend stats:
  → useEventService tracks conversation events (F15)
    → Mixpanel analytics for usage patterns (F15)
    → statsService reports token costs (F16)
```

Modules Involved: F15, F16 → B1, B2, B5

Tagged Files:

File	Role
`frontend/hooks/useEventService.ts`	Mixpanel analytics integration
`frontend/utils/app/stats.ts`	Token cost calculation
`backend/amplify-lambda-js/common/accounting.js`	Usage recording to DynamoDB
`backend/amplify-lambda-js/models/models.js`	Model pricing (input/output cost per token)
`backend/chat-billing/`	DynamoDB Stream consumer for cost calculation
`backend/amplify-lambda/accounts/accounts.py`	Account balance management
`frontend/services/accountService.ts`	Account API client

---

5. Module Specifications

5.1 Frontend: Chat Interface (F2)

Location: `components/Chat/`

Sub-components:

Component	File	Function
Chat	`Chat.tsx`	Main chat container; message rendering, scroll management, conversation controls
ChatInput	`ChatInput.tsx`	Message input textarea with file attachment, model selection, send button
ChatLoader	`ChatLoader.tsx`	Animated loading indicator during streaming
ChatMessage	`ChatMessage.tsx`	Individual message bubble with copy, edit, delete actions
MemoizedChatMessage	`MemoizedChatMessage.tsx`	Performance-optimized message wrapper (React.memo)
MemoizedRemoteMessages	`MemoizedRemoteMessages.tsx`	Cloud-stored message renderer
ModelSelect	`ModelSelect.tsx`	LLM model dropdown selector
SystemPrompt	`SystemPrompt.tsx`	Per-conversation system prompt editor
TemperatureSlider	`Temperature.tsx`	Temperature parameter slider (0-2)
ResponseTokensSlider	`ResponseTokens.tsx`	Max response tokens configuration
TagsList	`TagsList.tsx`	Conversation tagging UI
VariableModal	`VariableModal.tsx`	Prompt template variable input dialog
CloudStorage	`CloudStorage.tsx`	Cloud storage indicator and controls
ShareAnythingModal	(from `Share/`)	Conversation sharing dialog
DownloadModal	(from `Download/`)	Conversation export dialog
ErrorMessageDiv	`ErrorMessageDiv.tsx`	Error display component

State Dependencies:

- `selectedConversation` - Currently active conversation
- `messageIsStreaming` - Whether a response is being streamed
- `models` - Available LLM models
- `defaultModelId` - User's default model selection
- `featureFlags` - Feature toggle states

5.2 Frontend: Services Layer (F14)

Location: `services/`

Service	File	API Calls	Purpose
Chat	`chatService.ts`	POST chat endpoint	Streaming chat with SSE parsing, kill request
File	`fileService.ts`	`/files/upload`, `/files/download`, `/files/query`, `/files/setTags`	S3 presigned URLs, file metadata, tagging
Assistant	`assistantService.ts`	`/assistant/op`	Create, list, delete, run assistants
Remote Conversation	`remoteConversationService.ts`	`/conversation/remote/op`	Cloud conversation sync (upload, download, delete)
Share	`shareService.ts`	`/share/share`, `/share/shared`, `/share/youshared`	Conversation and item sharing
Market	`marketService.ts`	`/market/op`	Marketplace browse, publish, install
Base Prompts	`basePromptsService.ts`	`/state/base-prompts/get`	System prompt template retrieval
QI	`qiService.ts`	`/qi/upload`	Document summarization
Account	`accountService.ts`	`/accounts/op`	Multi-account management
Data Disclosure	`dataDisclosureService.ts`	`/datadisclosure/*`	Privacy compliance checks
Email Autocomplete	`emailAutocompleteService.ts`	`/emails/autocomplete`	User email suggestions
Ops	`opsService.ts`	`/ops/op`	Generic operation execution
State	`stateService.ts`	`/state`	State persistence
Download	`downloadService.ts`	N/A	File download management
Error	`errorService.ts`	N/A	Error handling and reporting

Service	File	API Calls	Purpose
API Helper	`useApiService.ts`	N/A	Shared fetch utilities
PDB	`pdbService.ts`	`/pdb/op`	Protein database integration

5.3 Backend: Chat Streaming Engine (B2)

Location: `amplify-lambda-js/`

Request Processing Pipeline:

Stage	File	Function
1. Entry	`index.js`	Lambda handler with HTTP/2 streamifyResponse
2. Parse	`common/params.js`	Extract JWT, body, headers, determine model
3. Route	`router.js`	Validate auth, check killswitch, resolve data sources
4. Model Select	`router.js`	Route to Azure OpenAI / Bedrock Claude / Bedrock Mistral
5. Assistant	`assistants/assistants.js`	Load assistant context if applicable
6. RAG	`common/chat/rag/rag.js`	Query embedding service for relevant chunks
7. Fit Context	`common/chatWithData.js`	Prune message history to fit token limit
8. LLM Call	`azure/openai.js` or `bedrock/*.js`	Stream LLM request
9. Transform	`common/chat/events/*.js`	Normalize response format across providers
10. Stream	`common/streams.js`	Write SSE events to response stream
11. Bill	`common/accounting.js`	Record token usage to DynamoDB
12. Trace	`common/trace.js`	Optional request tracing to S3
13. Cleanup	`requests/requestState.js`	Delete request state

Supported Models:

Provider	Models	Token Limits
Azure OpenAI	GPT-4 Turbo, GPT-4o, GPT-4o-mini, GPT-3.5 Turbo	4K - 128K
AWS Bedrock (Anthropic)	Claude Instant, Claude 2.1, Claude 3 Sonnet/Haiku/Opus, Claude 3.5 Sonnet	4K - 200K
AWS Bedrock (Mistral)	Mistral 7B, Mixtral 8x7B, Mistral Large	4K - 32K

5.4 Backend: RAG Pipeline (B1 + B4)

Document Processing Stages:

Stage	Lambda	Trigger	File	Output
1. Queue for RAG	`queue_document_for_rag`	S3 upload	`amplify-lambda/rag/core.py`	SQS message
2. Extract Text	`process_document_for_rag`	SQS	`amplify-lambda/rag/handlers/*.py`	Text in S3
3. Queue for Chunking	`queue_document_for_chunking`	S3 event	`amplify-lambda/rag/core.py`	SQS message
4. Chunk Document	`process_document_for_chunking`	SQS	`amplify-lambda/rag/chunk/*.py`	Chunks in S3
5. Queue for Embedding	`queue_document_for_embedding`	S3 event	`embedding/embedding-sqs.py`	SQS message
6. Generate Embedding	`process_chunk_for_embedding`	SQS (200 workers)	`embedding/embedding.py`	Vectors in PostgreSQL
7. Search/ Retrieve	`get_dual_embeddings`	API call	`embedding/embedding-dual-retrieval.py`	Ranked results

Supported Document Formats:

Format	Handler	Library
PDF	`rag/handlers/pdf.py`	pypdfium2
DOCX	`rag/handlers/docx.py`	python-docx
PPTX	`rag/handlers/pptx.py`	python-pptx
XLSX/XLS	`rag/handlers/xlsx.py`	openpyxl
CSV	`rag/handlers/csv.py`	built-in
TXT/HTML	`rag/handlers/text.py`	chardet, beautifulsoup4

5.5 Backend: Object Access Control (B6)

Location: `object-access/`

API Endpoints:

Endpoint	Method	Function
`/utilities/emails`	GET	List Cognito users for permission grants
`/utilities/update_object_permissions`	POST	Create/update/revoke object permissions
`/utilities/can_access_objects`	POST	Check if user can access specific objects
`/utilities/simulate_access_to_objects`	POST	Preview permission changes before applying

Permission Model:
- Per-object ACLs stored in DynamoDB
- Object types: assistants, files, prompts, conversations
- Permission levels: read, write, admin
- Cognito user pool integration for user resolution

---

6. Cross-Module Integration Map

```
Frontend                          Backend
────────                          ───────

                     ┌──────────────────────────────────┐
F2 Chat ────────────▶│ B2 amplify-lambda-js (streaming) │
F15 Hooks            │   ├─▶ Azure OpenAI (GPT models)  │
F14 chatService      │   ├─▶ Bedrock Claude             │
                     │   ├─▶ Bedrock Mistral            │
                     │   ├─▶ B4 embedding (RAG queries) │
                     │   ├─▶ B5 chat-billing (usage)    │
                     │   └─▶ B6 object-access (perms)   │
                     └──────────────────────────────────┘

F6 Assistants ──────▶│ B1 amplify-lambda (CRUD)         │
F9 DataSources       │   ├─▶ S3 (file storage)          │
F14 fileService      │   ├─▶ DynamoDB (metadata)        │
F14 assistantSvc     │   └─▶ B6 object-access (perms)   │
                     └──────────────────────────────────┘

F6 Assistants ──────▶│ B3 amplify-assistants            │
                     │   ├─▶ OpenAI Assistants API      │
                     │   └─▶ B6 object-access (perms)   │
                     └──────────────────────────────────┘

F7 Market ──────────▶│ B1 amplify-lambda (marketplace)  │
F14 marketService    │   ├─▶ DynamoDB (items, categories) │
                     │   └─▶ S3 (search index)          │
                     └──────────────────────────────────┘

F8 Share ───────────▶│ B1 amplify-lambda (sharing)      │
F14 shareService     │   ├─▶ S3 (shared state)          │
                     │   └─▶ B6 object-access (perms)   │
                     └──────────────────────────────────┘

F9 DataSources ─────▶│ B1 amplify-lambda (upload)       │──────▶ B4 embedding
F14 fileService      │   ├─▶ S3 (raw documents)         │       (async via S3→SQS)
                     │   ├─▶ SQS (processing queue)     │
                     │   └─▶ Text extraction handlers   │
                     └──────────────────────────────────┘
```

7. Data Model

7.1 Core Entities

Conversation

```
{
  id: string                          // UUID
  name: string                        // Display name (auto-generated or user-set)
  messages: Message[]                 // Array of chat messages
  compressedMessages?: number[]       // LZW-compressed messages for storage
  model: OpenAIModel                  // Selected LLM model
  prompt?: string                     // System prompt override
  temperature?: number                // Temperature parameter (0-2)
  folderId: string | null             // Folder organization
  tags?: string[]                     // User-defined tags
  workflowDefinition?: WorkflowDefinition  // Optional workflow
  isRemote?: boolean                  // Cloud-stored flag
}
```

Message

```
{
  id: string                  // UUID
  role: 'user' | 'assistant' | 'system'
  content: string             // Message text
  type: MessageType           // Message format type
  data: any                   // Structured data (JSON, CSV, etc.)
  label?: string              // Display label
}
```

Assistant

```
{
  id: string                  // UUID
  name: string                // Display name
  description: string         // Description
  instructions: string        // System prompt for assistant
  tools: AssistantTool[]      // Enabled tools (code_interpreter, retrieval)
  fileKeys: string[]          // Attached file S3 keys
  dataSources: AttachedDocument[]  // RAG data sources
  provider: 'openai' | 'amplify'  // Assistant platform
  uri?: string                // External URI
  data?: any                  // Additional config
}
```

File Record

```
{
  id: string                  // UUID
  name: string                // Original filename
  type: string                // MIME type
  tags: string[]              // User-defined tags
  totalTokens?: number        // Token count
  createdAt: string           // ISO timestamp
  updatedAt: string           // ISO timestamp
  createdBy: string           // Owner user ID
}
```

Workflow Definition

```
{
  id: string                    // UUID
  name: string                  // Display name
  code: string                  // Workflow logic (YAML/JSON)
  inputs: Inputs                // Input schema
  outputs: OutputType[]         // Output type definitions
  folderId: string | null       // Folder organization
}
```

7.2 Storage Summary

Store	Technology	Data
Browser LocalStorage	IndexedDB/ LocalStorage	Conversations (compressed), settings, folders, prompts
S3 State Bucket	Amazon S3	Cloud conversations, shared items, base prompts
S3 File Buckets	Amazon S3	Uploaded documents, RAG chunks, extracted text, traces
DynamoDB Tables	Amazon DynamoDB	Assistants, threads, files, tags, accounts, billing, market items, permissions
PostgreSQL pgvector	Aurora PostgreSQL	Vector embeddings for RAG semantic search
Secrets Manager	AWS Secrets Manager	API keys, DB credentials, JWT secrets

---

8. File-to-Functionality Tag Map

Frontend (`amplify-genai-frontend/`)

Authentication & Session

File	Tag
`pages/api/auth/[...nextauth].js`	`AUTH` , `SESSION` , `COGNITO`
`pages/_app.tsx`	`AUTH` , `APP_SHELL`

App Shell & State

File	Tag
`pages/api/home/home.tsx`	`APP_SHELL`, `STATE`, `INIT`
`pages/api/home/home.context.tsx`	`STATE`, `CONTEXT`
`pages/api/home/home.state.tsx`	`STATE`, `DEFAULTS`

Chat

File	Tag
`components/Chat/Chat.tsx`	`CHAT`, `UI`, `MESSAGES`
`components/Chat/ChatInput.tsx`	`CHAT`, `INPUT`, `FILE_ATTACH`
`components/Chat/ChatMessage.tsx`	`CHAT`, `MESSAGE_RENDER`
`components/Chat/MemoizedChatMessage.tsx`	`CHAT`, `PERFORMANCE`
`components/Chat/MemoizedRemoteMessages.tsx`	`CHAT`, `CLOUD_SYNC`
`components/Chat/ModelSelect.tsx`	`CHAT`, `MODEL_SELECT`
`components/Chat/SystemPrompt.tsx`	`CHAT`, `SYSTEM_PROMPT`
`components/Chat/Temperature.tsx`	`CHAT`, `CONFIG`
`components/Chat/ResponseTokens.tsx`	`CHAT`, `CONFIG`
`components/Chat/TagsList.tsx`	`CHAT`, `TAGS`
`components/Chat/VariableModal.tsx`	`CHAT`, `PROMPTS`, `TEMPLATES`
`components/Chat/CloudStorage.tsx`	`CHAT`, `CLOUD_SYNC`
`components/Chat/ChatLoader.tsx`	`CHAT`, `UI`
`components/Chat/ErrorMessageDiv.tsx`	`CHAT`, `ERROR_HANDLING`

Sidebars & Navigation

File	Tag
`components/Chatbar/Chatbar.tsx`	`SIDEBAR`, `CONVERSATIONS`, `FOLDERS`
`components/Promptbar/`	`SIDEBAR`, `PROMPTS`, `TEMPLATES`
`components/TabSidebar/TabSidebar.tsx`	`NAVIGATION`, `TABS`
`components/Sidebar/`	`SIDEBAR`, `UI`
`components/Mobile/Navbar.tsx`	`MOBILE`, `NAVIGATION`

Assistants

File	Tag
`components/Assistants/`	`ASSISTANTS`, `UI`, `CONFIG`
`services/assistantService.ts`	`ASSISTANTS`, `API_CLIENT`
`types/assistant.ts`	`ASSISTANTS`, `TYPES`
`utils/app/assistants.ts`	`ASSISTANTS`, `UTILS`

Marketplace

File	Tag
`components/Market/Market.tsx`	`MARKETPLACE`, `UI`, `BROWSE`
`services/marketService.ts`	`MARKETPLACE`, `API_CLIENT`
`types/market.ts`	`MARKETPLACE`, `TYPES`

Sharing

File	Tag
`components/Share/ShareAnythingModal.tsx`	`SHARING`, `UI`, `MODAL`
`components/Share/SharedItemList.tsx`	`SHARING`, `UI`, `LIST`
`services/shareService.ts`	`SHARING`, `API_CLIENT`

Data Sources / Files

File	Tag
`components/DataSources/`	`DATA_SOURCES`, `RAG`, `UI`
`services/fileService.ts`	`FILES`, `S3`, `API_CLIENT`

Settings & Workspace

File	Tag
`components/Settings/SettingsBar.tsx`	`SETTINGS`, `UI`
`components/Workspace/WorkspaceList.tsx`	`WORKSPACE`, `UI`

Export / Download

File	Tag
`components/Download/DownloadModal.tsx`	`EXPORT`, `UI`
`services/downloadService.ts`	`EXPORT`, `SERVICE`

API Proxy Layer

File	Tag
`pages/api/assistant/op.ts`	`API_PROXY`, `ASSISTANTS`
`pages/api/files/upload.ts`	`API_PROXY`, `FILES`
`pages/api/files/download.ts`	`API_PROXY`, `FILES`
`pages/api/files/query.ts`	`API_PROXY`, `FILES`
`pages/api/files/setTags.ts`	`API_PROXY`, `FILES`, `TAGS`
`pages/api/share/share.ts`	`API_PROXY`, `SHARING`
`pages/api/share/shared.ts`	`API_PROXY`, `SHARING`
`pages/api/market/op.ts`	`API_PROXY`, `MARKETPLACE`
`pages/api/state.ts`	`API_PROXY`, `STATE`
`pages/api/conversation/remote/op.ts`	`API_PROXY`, `CLOUD_SYNC`
`pages/api/accounts/op.ts`	`API_PROXY`, `BILLING`
`pages/api/ops/op.ts`	`API_PROXY`, `OPERATIONS`
`pages/api/baseprompts/op.ts`	`API_PROXY`, `PROMPTS`
`pages/api/emails/autocomplete.ts`	`API_PROXY`, `SHARING`
`pages/api/cognito_groups/op.ts`	`API_PROXY`, `AUTH`
`pages/api/datadisclosure/*.ts`	`API_PROXY`, `COMPLIANCE`
`pages/api/google.ts`	`API_PROXY`, `SEARCH`
`pages/api/qi/upload.ts`	`API_PROXY`, `QI`
`pages/api/pdb/op.ts`	`API_PROXY`, `PDB`

Services

File	Tag
`services/chatService.ts`	`CHAT`, `STREAMING`, `API_CLIENT`
`services/remoteConversationService.ts`	`CLOUD_SYNC`, `API_CLIENT`
`services/basePromptsService.ts`	`PROMPTS`, `API_CLIENT`
`services/qiService.ts`	`QI`, `API_CLIENT`
`services/accountService.ts`	`BILLING`, `API_CLIENT`
`services/dataDisclosureService.ts`	`COMPLIANCE`, `API_CLIENT`
`services/emailAutocompleteService.ts`	`SHARING`, `API_CLIENT`
`services/opsService.ts`	`OPERATIONS`, `API_CLIENT`
`services/stateService.ts`	`STATE`, `API_CLIENT`
`services/errorService.ts`	`ERROR_HANDLING`, `SERVICE`
`services/useApiService.ts`	`API_UTILS`, `SERVICE`
`services/pdbService.ts`	`PDB`, `API_CLIENT`

Hooks

File	Tag
`hooks/useChatSendService.ts`	`CHAT`, `ROUTING`, `STRUCTURED_OUTPUT`
`hooks/useChatService.ts`	`CHAT`, `LLM_CALLS`, `FUNCTION_CALLING`
`hooks/useHomeReducer.ts`	`STATE`, `REDUCER`
`hooks/useEventService.ts`	`ANALYTICS`, `MIXPANEL`
`hooks/useOpsService.ts`	`WORKFLOWS`, `OPERATIONS`
`hooks/usePromptFinderService.ts`	`PROMPTS`, `PREFIX_DETECTION`
`hooks/useFetch.ts`	`API_UTILS`, `FETCH`
`hooks/useCreateReducer.ts`	`STATE`, `UTILS`

Types

File	Tag
`types/chat.ts`	`TYPES`, `CHAT`, `MESSAGES`
`types/openai.ts`	`TYPES`, `MODELS`, `PRICING`
`types/assistant.ts`	`TYPES`, `ASSISTANTS`
`types/attacheddocument.ts`	`TYPES`, `FILES`, `RAG`
`types/workflow.ts`	`TYPES`, `WORKFLOWS`
`types/prompt.ts`	`TYPES`, `PROMPTS`
`types/folder.ts`	`TYPES`, `FOLDERS`
`types/market.ts`	`TYPES`, `MARKETPLACE`
`types/export.ts`	`TYPES`, `EXPORT`
`types/tags.ts`	`TYPES`, `TAGS`
`types/plugin.ts`	`TYPES`, `SEARCH`
`types/data.ts`	`TYPES`, `UTILS`
`types/qi.ts`	`TYPES`, `QI`

Utilities

File	Tag
`utils/app/conversation.ts`	`UTILS`, `CONVERSATIONS`, `COMPRESSION`
`utils/app/conversationStorage.ts`	`UTILS`, `STORAGE`, `CLOUD_SYNC`
`utils/app/messages.ts`	`UTILS`, `COMPRESSION`
`utils/app/lzwCompression.ts`	`UTILS`, `COMPRESSION`, `ALGORITHM`
`utils/app/prompts.ts`	`UTILS`, `PROMPTS`, `TEMPLATES`
`utils/app/folders.ts`	`UTILS`, `FOLDERS`
`utils/app/workflows.ts`	`UTILS`, `WORKFLOWS`
`utils/app/settings.ts`	`UTILS`, `SETTINGS`
`utils/app/openai.ts`	`UTILS`, `TOKENS`, `COUNTING`
`utils/app/assistants.ts`	`UTILS`, `ASSISTANTS`
`utils/app/state.ts`	`UTILS`, `STATE`
`utils/app/api.ts`	`UTILS`, `ROUTING`
`utils/app/const.ts`	`UTILS`, `CONSTANTS`
`utils/app/chathooks.ts`	`UTILS`, `CHAT`, `PREPROCESSING`
`utils/app/csv.ts`	`UTILS`, `CSV`, `STRUCTURED_OUTPUT`
`utils/app/date.ts`	`UTILS`, `DATE`
`utils/app/stats.ts`	`UTILS`, `ANALYTICS`, `BILLING`
`utils/app/codeblock.ts`	`UTILS`, `RENDERING`
`utils/app/features.ts`	`UTILS`, `FEATURE_FLAGS`
`utils/app/clean.ts`	`UTILS`, `SANITIZATION`
`utils/app/importExport.ts`	`UTILS`, `EXPORT`, `IMPORT`
`utils/app/outOfOrder.ts`	`UTILS`, `STREAMING`
`utils/app/responseWrapper.ts`	`UTILS`, `STREAMING`, `SSE`
`utils/app/incrementalCsvParser.ts`	`UTILS`, `CSV`, `STREAMING`

Backend (`amplify-genai-backend/`)

amplify-lambda (B1) - Core REST API

File	Tag
`amplify-lambda/assistant/assistant.py`	`FILES`, `ASSISTANTS`, `THREADS`, `TAGS`, `PRESIGNED_URL`
`amplify-lambda/assistant/assistant_api.py`	`FILES`, `DYNAMODB`, `DATA_LAYER`
`amplify-lambda/accounts/accounts.py`	`BILLING`, `ACCOUNTS`
`amplify-lambda/market/market.py`	`MARKETPLACE`, `CRUD`, `SEARCH`
`amplify-lambda/market/market_ideator.py`	`MARKETPLACE`, `AI_GENERATION`
`amplify-lambda/market/rebuild_index.py`	`MARKETPLACE`, `INDEXING`
`amplify-lambda/personal/assistant.py`	`EMAIL_ASSISTANT`, `SNS`
`amplify-lambda/rag/core.py`	`RAG`, `PIPELINE`, `ORCHESTRATION`
`amplify-lambda/rag/util.py`	`RAG`, `UTILS`
`amplify-lambda/rag/handlers/pdf.py`	`RAG`, `TEXT_EXTRACTION`, `PDF`
`amplify-lambda/rag/handlers/docx.py`	`RAG`, `TEXT_EXTRACTION`, `DOCX`
`amplify-lambda/rag/handlers/pptx.py`	`RAG`, `TEXT_EXTRACTION`, `PPTX`
`amplify-lambda/rag/handlers/xlsx.py`	`RAG`, `TEXT_EXTRACTION`, `XLSX`
`amplify-lambda/rag/handlers/csv.py`	`RAG`, `TEXT_EXTRACTION`, `CSV`
`amplify-lambda/rag/handlers/text.py`	`RAG`, `TEXT_EXTRACTION`, `TXT`
`amplify-lambda/rag/chunk/pdf.py`	`RAG`, `CHUNKING`, `PDF`
`amplify-lambda/rag/chunk/docx.py`	`RAG`, `CHUNKING`, `DOCX`
`amplify-lambda/rag/chunk/pptx.py`	`RAG`, `CHUNKING`, `PPTX`
`amplify-lambda/rag/chunk/xlsx.py`	`RAG`, `CHUNKING`, `XLSX`
`amplify-lambda/rag/chunk/csv.py`	`RAG`, `CHUNKING`, `CSV`
`amplify-lambda/rag/chunk/html.py`	`RAG`, `CHUNKING`, `HTML`
`amplify-lambda/rag/chunk/text.py`	`RAG`, `CHUNKING`, `TXT`
`amplify-lambda/converters/docconverter.py`	`DOC_CONVERSION`, `PANDOC`
`amplify-lambda/state/create.py`	`STATE`, `PERSISTENCE`
`amplify-lambda/state/share.py`	`SHARING`, `STATE`, `S3`
`amplify-lambda/common/validate.py`	`AUTH`, `VALIDATION`, `MIDDLEWARE`
`amplify-lambda/common/permissions.py`	`AUTH`, `RBAC`

File	Tag
`amplify-lambda/common/object_permissions.py`	`AUTH`, `ACL`
`amplify-lambda/common/data_sources.py`	`DATA_SOURCES`, `RESOLUTION`
`amplify-lambda/common/share_assistants.py`	`SHARING`, `ASSISTANTS`
`amplify-lambda/common/secrets.py`	`SECRETS`, `CONFIG`
`amplify-lambda/common/encoders.py`	`UTILS`, `SERIALIZATION`
`amplify-lambda/admins/market_creator.py`	`ADMIN`, `MARKETPLACE`
`amplify-lambda/admins/market_item_loader.py`	`ADMIN`, `MARKETPLACE`
`amplify-lambda/serverless.yml`	`IAC`, `DEPLOYMENT`, `RESOURCES`

amplify-lambda-js (B2) - Streaming Chat

File	Tag
`amplify-lambda-js/index.js`	`CHAT`, `ENTRY_POINT`, `HTTP2_STREAMING`
`amplify-lambda-js/router.js`	`CHAT`, `ROUTING`, `AUTH`, `MODEL_SELECT`
`amplify-lambda-js/common/llm.js`	`CHAT`, `LLM_ABSTRACTION`
`amplify-lambda-js/common/chatWithData.js`	`CHAT`, `RAG`, `CONTEXT_INJECTION`
`amplify-lambda-js/common/permissions.js`	`AUTH`, `DATA_SOURCE_ACCESS`
`amplify-lambda-js/common/multiplexer.js`	`CHAT`, `MULTI_SOURCE`
`amplify-lambda-js/common/accounting.js`	`BILLING`, `TOKEN_TRACKING`
`amplify-lambda-js/common/secrets.js`	`SECRETS`, `CONFIG`
`amplify-lambda-js/common/streams.js`	`STREAMING`, `SSE`, `EVENTS`
`amplify-lambda-js/common/logging.js`	`LOGGING`, `OBSERVABILITY`
`amplify-lambda-js/common/trace.js`	`TRACING`, `OBSERVABILITY`
`amplify-lambda-js/common/params.js`	`PARSING`, `JWT`, `REQUEST`
`amplify-lambda-js/common/status.js`	`STREAMING`, `STATUS_EVENTS`
`amplify-lambda-js/common/incrementalJsonParser.js`	`STREAMING`, `JSON_PARSING`
`amplify-lambda-js/common/sources.js`	`RAG`, `SOURCE_TRACKING`
`amplify-lambda-js/common/chat/controllers/sequentialChat.js`	`CHAT`, `SEQUENTIAL`
`amplify-lambda-js/common/chat/controllers/parallelChat.js`	`CHAT`, `PARALLEL`
`amplify-lambda-js/common/chat/controllers/meta.js`	`CHAT`, `METADATA`
`amplify-lambda-js/common/chat/events/openai.js`	`CHAT`, `OPENAI`, `EVENT_TRANSFORM`
`amplify-lambda-js/common/chat/events/bedrock.js`	`CHAT`, `BEDROCK`, `EVENT_TRANSFORM`
`amplify-lambda-js/common/chat/events/openaifn.js`	`CHAT`, `FUNCTION_CALLING`
`amplify-lambda-js/common/chat/rag/rag.js`	`RAG`, `QUERY`, `RETRIEVAL`
`amplify-lambda-js/models/models.js`	`MODELS`, `PRICING`, `REGISTRY`
`amplify-lambda-js/azure/openai.js`	`LLM_PROVIDER`, `AZURE_OPENAI`
`amplify-lambda-js/azure/tokens.js`	`TOKENS`, `COUNTING`, `TIKTOKEN`
`amplify-lambda-js/bedrock/anthropic.js`	`LLM_PROVIDER`, `BEDROCK`, `CLAUDE`

File	Tag
`amplify-lambda-js/bedrock/mistral.js`	`LLM_PROVIDER`, `BEDROCK`, `MISTRAL`
`amplify-lambda-js/datasource/datasources.js`	`DATA_SOURCES`, `RESOLUTION`, `ACL`
`amplify-lambda-js/assistants/assistants.js`	`ASSISTANTS`, `CHAT_ROUTING`
`amplify-lambda-js/requests/requestState.js`	`KILLSWITCH`, `REQUEST_STATE`
`amplify-lambda-js/workflow/workflow.js`	`WORKFLOWS`, `MULTI_STEP`, `ENGINE`
`amplify-lambda-js/workflow/workflowCreator.js`	`WORKFLOWS`, `VALIDATION`
`amplify-lambda-js/assistantQueueRouter.js`	`ASSISTANTS`, `QUEUE_ROUTING`
`amplify-lambda-js/serverless.yml`	`IAC`, `DEPLOYMENT`, `RESOURCES`

amplify-assistants (B3) - OpenAI Assistants

File	Tag
`amplify-assistants/service/core.py`	`ASSISTANTS`, `CORE_LOGIC`
`amplify-assistants/openaiazure/assistant.py`	`ASSISTANTS`, `OPENAI_API`
`amplify-assistants/openaiazure/assistant_api.py`	`ASSISTANTS`, `API_CLIENT`
`amplify-assistants/openaiazure/token.py`	`ASSISTANTS`, `TOKENS`
`amplify-assistants/common/validate.py`	`AUTH`, `VALIDATION`
`amplify-assistants/common/permissions.py`	`AUTH`, `RBAC`
`amplify-assistants/common/object_permissions.py`	`AUTH`, `ACL`
`amplify-assistants/common/data_sources.py`	`DATA_SOURCES`
`amplify-assistants/common/secrets.py`	`SECRETS`
`amplify-assistants/common/credentials.py`	`AUTH`, `AWS_CREDS`
`amplify-assistants/common/encoders.py`	`UTILS`, `SERIALIZATION`
`amplify-assistants/serverless.yml`	`IAC`, `DEPLOYMENT`

embedding (B4) - Vector Embeddings & RAG Search

File	Tag
`embedding/embedding.py`	`EMBEDDING`, `VECTOR_GENERATION`, `SQS_WORKER`
`embedding/embedding-sqs.py`	`EMBEDDING`, `QUEUE`, `S3_TRIGGER`
`embedding/embedding-dual-retrieval.py`	`EMBEDDING`, `SEARCH`, `HYBRID_RETRIEVAL`
`embedding/shared_functions.py`	`EMBEDDING`, `UTILS`
`embedding/create_table.py`	`EMBEDDING`, `SCHEMA`, `PGVECTOR`
`embedding/common/validate.py`	`AUTH`, `VALIDATION`
`embedding/common/permissions.py`	`AUTH`, `RBAC`
`embedding/common/encoders.py`	`UTILS`, `SERIALIZATION`
`embedding/common/credentials.py`	`AUTH`, `AWS_CREDS`
`embedding/serverless.yml`	`IAC`, `DEPLOYMENT`, `VPC`, `RDS`

chat-billing (B5) - Usage & Billing

File	Tag
`chat-billing/` (all files)	`BILLING`, `USAGE_TRACKING`, `EXCHANGE_RATES`
`chat-billing/serverless.yml`	`IAC`, `DEPLOYMENT`, `DYNAMO_STREAMS`

object-access (B6) - Permissions

File	Tag
`object-access/object_access.py`	`PERMISSIONS`, `RBAC`, `ACL`
`object-access/cognito_users.py`	`PERMISSIONS`, `USER_LOOKUP`, `COGNITO`
`object-access/serverless.yml`	`IAC`, `DEPLOYMENT`

utilities (B7) - Misc Operations

File	Tag
`utilities/rename_chats.py`	`UTILS`, `BATCH_OPS`, `RENAME`
`utilities/serverless.yml`	`IAC`, `DEPLOYMENT`

Infrastructure

File	Tag
`serverless-compose.yml`	`IAC`, `ORCHESTRATION`, `ROOT`
`package.json`	`IAC`, `ROOT_DEPS`
`dev-var.yml-example`	`IAC`, `ENV_CONFIG`, `TEMPLATE`

Appendix A: Feature Flags

Flag	Default	Controls
`assistantsEnabled`	`true`	Show assistants panel
`ragEnabled`	`true`	Enable RAG/document query
`sourcesEnabled`	`true`	Show source citations
`uploadDocuments`	`true`	Allow file uploads
`assistantCreator`	`true`	Allow creating custom assistants
`rootPromptCreate`	`true`	Allow creating root prompts
`followUpCreate`	`true`	Allow follow-up prompt creation
`workflowRun`	`true`	Allow running workflows
`automation`	`true`	Enable automation features
`dataSourceSelectorOnInput`	`true`	Show data source selector in chat input

Appendix B: Environment Variables Reference

Frontend (.env.local)

Variable	Purpose
`NEXTAUTH_SECRET`	NextAuth session encryption
`NEXTAUTH_URL`	Application base URL
`COGNITO_CLIENT_ID`	AWS Cognito app client ID
`COGNITO_CLIENT_SECRET`	AWS Cognito app client secret
`COGNITO_ISSUER`	Cognito user pool domain
`API_BASE_URL`	Backend API Gateway base URL
`DEFAULT_MODEL`	Default LLM model ID
`AVAILABLE_MODELS`	Comma-separated available model IDs
`NEXT_PUBLIC_MIXPANEL_TOKEN`	Mixpanel analytics token
`GOOGLE_API_KEY`	Google Search API key
`GOOGLE_CSE_ID`	Google Custom Search Engine ID

Backend (dev-var.yml)

Variable	Purpose
`OAUTH_AUDIENCE`	JWT audience claim
`OAUTH_ISSUER_BASE_URL`	Cognito domain URL
`COGNITO_USER_POOL_ID`	Cognito user pool
`LLM_ENDPOINTS_SECRETS_NAME`	Secrets Manager key for LLM API keys
`OPENAI_PROVIDER`	LLM provider (`openai` or `azure`)
`EMBEDDING_PROVIDER`	Embedding provider (`openai` or `bedrock`)
`EMBEDDING_MODEL_NAME`	Embedding model ID
`VPC_ID`	VPC for Aurora PostgreSQL
`CUSTOM_API_DOMAIN`	Custom API domain name

Generated by automated codebase analysis. Amplify GenAI Platform - Vanderbilt University.